## beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

## data loading

```
df_non_encoded = pd.read_excel("/content/drive/MyDrive/capstone/datasets/4_non_encoded.xlsx")
```

```
df_non_encoded.tail()
```

| | age | age_group | gender | marital_status | profession_group | workplace | religion | financial_condition | hometown | latitude | ... | wind_speed | wind_deg | clouds_sky | weather_mai |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **1612** | 15.0 | teen | female | relationship | student | rural | muslim | NaN | NaN | NaN | ... | NaN | NaN | NaN | Na |
| **1613** | 27.0 | adult | female | single | student | urban | muslim | NaN | NaN | NaN | ... | NaN | NaN | NaN | Na |
| **1614** | 9.0 | child | male | NaN | NaN | NaN | muslim | NaN | NaN | NaN | ... | NaN | NaN | NaN | Na |
| **1615** | NaN | young | female | NaN | student | rural | muslim | NaN | kushtia | 23.898795 | ... | 8.0 | NaN | NaN | Na |
| **1616** | 26.0 | adult | female | married | housewife | NaN | NaN | NaN | bagerhat | 22.655478 | ... | 3.8 | NaN | NaN | Na |

5 rows × 29 columns

```
df_non_encoded.shape
```

```
(1617, 29)
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Get the data types of each column
```

```
    data_types = df.dtypes

    # Create a DataFrame for the missing data summary including data types
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values,
        'Data Type': data_types.values
    })

    # Define the four groups based on missing percentage
    group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage', 'Data Type']]
    group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage'
    group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage'
    group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage', 'Data Type']]

    # Sort each group by missing percentage in descending order (most missing to least missing)
    group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)


    # Rename columns for clarity
    group_1.columns = ['Column', 'Missing %', 'Data Type']
    group_2.columns = ['Column', 'Missing %', 'Data Type']
    group_3.columns = ['Column', 'Missing %', 'Data Type']
    group_4.columns = ['Column', 'Missing %', 'Data Type']

    # Display each group one by one, from most missing to least missing
    print("Group 1: Columns with >70% missing data")
    print(group_1.to_string(index=False))
    print("\nGroup 2: Columns with >40% to ≤70% missing data")
    print(group_2.to_string(index=False))
    print("\nGroup 3: Columns with >20% to ≤40% missing data")
    print(group_3.to_string(index=False))
    print("\nGroup 4: Columns with ≤20% missing data")
    print(group_4.to_string(index=False))
```

```
missing_data_report(df_non_encoded)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

Group 2: Columns with >40% to ≤70% missing data
             Column  Missing % Data Type
         feels_like  53.061224   float64
       air_humidity  53.061224   float64
           wind_deg  53.061224   float64
         clouds_sky  53.061224   float64
       weather_main  52.999382    object
weather_description  52.999382    object
financial_condition  45.516388    object

Group 3: Columns with >20% to ≤40% missing data
        Column  Missing % Data Type
```

```
     workplace  37.909709    object
  air_pressure  30.426716   float64
    wind_speed  30.179344   float64
      temp_min  29.870130   float64
      temp_max  29.560915   float64
marital_status  23.933210    object

Group 4: Columns with ≤20% missing data
          Column  Missing % Data Type
profession_group  18.923933    object
          reason  16.821274    object
             age  12.739641   float64
            time  12.059369    object
       age_group   7.421150    object
       longitude   6.493506   float64
        latitude   6.493506   float64
          method   2.906617    object
   suicide_month   2.411874   float64
    suicide_year   2.411874   float64
        religion   2.411874    object
     suicide_day   2.411874   float64
    suicide_week   2.411874   float64
 suicide_weekday   2.411874    object
        hometown   1.669759    object
          gender   0.432900    object
```

```
df_non_encoded.columns
```

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
       'workplace', 'religion', 'financial_condition', 'hometown', 'latitude',
       'longitude', 'reason', 'time', 'method', 'feels_like', 'temp_min',
       'temp_max', 'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg',
       'clouds_sky', 'weather_main', 'weather_description', 'suicide_day',
       'suicide_month', 'suicide_year', 'suicide_week', 'suicide_weekday'],
      dtype='object')
```

```
df_non_encoded.head()
```

| | age | age_group | gender | marital_status | profession_group | workplace | religion | financial_condition | hometown | latitude | ... | wind_speed | wind_deg | clouds_sky | weather_mai |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | NaN | teen | male | NaN | worker | rural | muslim | indebted | brahmanbaria | 23.960600 | ... | 2.10 | 320.0 | 40.0 | haz |
| **1** | 20.0 | teen | male | rejected | NaN | NaN | muslim | NaN | sylhet | 24.899220 | ... | 1.39 | 116.0 | 33.0 | cloud |
| **2** | 14.0 | teen | female | single | student | urban | muslim | NaN | manikganj | 23.832984 | ... | 3.60 | 190.0 | 75.0 | drizzl |
| **3** | NaN | NaN | female | rejected | student | urban | muslim | NaN | bogura | 24.850066 | ... | 2.64 | 186.0 | 100.0 | rai |
| **4** | 18.0 | teen | female | relationship | student | urban | muslim | NaN | dhaka | 23.764386 | ... | 1.99 | 154.0 | 40.0 | haz |

5 rows × 29 columns

```
# from sklearn.preprocessing import LabelEncoder

# def label_encode_object_columns(df):
#     df_encoded = df.copy()
#     label_encoders = {}
```

```
#     for col in df_encoded.select_dtypes(include='object').columns:
#         le = LabelEncoder()
#         df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))
#         label_encoders[col] = le  # Store encoder if you want to inverse transform later

#     return df_encoded, label_encoders


from sklearn.preprocessing import LabelEncoder
import numpy as np
import pandas as pd

def label_encode_object_columns(df):
    df_encoded = df.copy()
    label_encoders = {}

    for col in df_encoded.select_dtypes(include='object').columns:
        le = LabelEncoder()
        non_null = df_encoded[col][df_encoded[col].notna()]  # only non-NaN values
        le.fit(non_null)
        # Encode non-NaN values, leave NaN as-is
        df_encoded[col] = df_encoded[col].map(lambda x: le.transform([x])[0] if pd.notna(x) else np.nan)
        label_encoders[col] = le

    return df_encoded, label_encoders
```

```
df_encoded, label_encoders = label_encode_object_columns(df_non_encoded)
```

```
df_encoded.head()
```

| | age | age_group | gender | marital_status | profession_group | workplace | religion | financial_condition | hometown | latitude | ... | wind_speed | wind_deg | clouds_sky | weather_main |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | NaN | 5.0 | 1.0 | NaN | 34.0 | 0.0 | 5.0 | 0.0 | 18.0 | 23.960600 | ... | 2.10 | 320.0 | 40.0 | 3.0 |
| **1** | 20.0 | 5.0 | 1.0 | 4.0 | NaN | NaN | 5.0 | NaN | 143.0 | 24.899220 | ... | 1.39 | 116.0 | 33.0 | 1.0 |
| **2** | 14.0 | 5.0 | 0.0 | 6.0 | 26.0 | 2.0 | 5.0 | NaN | 86.0 | 23.832984 | ... | 3.60 | 190.0 | 75.0 | 2.0 |
| **3** | NaN | NaN | 0.0 | 4.0 | 26.0 | 2.0 | 5.0 | NaN | 17.0 | 24.850066 | ... | 2.64 | 186.0 | 100.0 | 6.0 |
| **4** | 18.0 | 5.0 | 0.0 | 5.0 | 26.0 | 2.0 | 5.0 | NaN | 31.0 | 23.764386 | ... | 1.99 | 154.0 | 40.0 | 3.0 |

5 rows × 29 columns

```
import json
from sklearn.preprocessing import LabelEncoder
import os

# Prepare label_encoders for JSON serialization
# We'll store the classes_ attribute of each LabelEncoder
label_encoders_serializable = {
    col: list(le.classes_)
    for col, le in label_encoders.items()
}
```

```
# Specify the filename and the Google Drive path
drive_path = '/content/drive/MyDrive/capstone/' # Make sure this path exists in your Drive
filename = 'label_encoders.json'
full_path = os.path.join(drive_path, filename)

# Write to JSON file
with open(full_path, 'w') as f:
    json.dump(label_encoders_serializable, f, indent=4)

print(f"Label encoders saved to {full_path}")
```

```
Label encoders saved to /content/drive/MyDrive/capstone/label_encoders.json
```

```
import pandas as pd

def compute_sorted_correlations(df, threshold=0.0):
    corr_matrix = df.corr()

    result = []

    for col in corr_matrix.columns:
        # Drop self-correlation
        related = corr_matrix[col].drop(labels=[col])

        # Keep only correlations above threshold (excluding exact 0)
        related_filtered = related[related.abs() > threshold]

        # Sort by absolute correlation descending
        related_sorted = related_filtered.reindex(related_filtered.abs().sort_values(ascending=False).index)

        # Format results
        related_list = [
            {'related_column': other_col, 'related_value': round(related_value, 4)}
            for other_col, related_value in related_sorted.items()
        ]

        result.append({
            'column_name': col,
            'related_columns': related_list
        })

    return result
```

```
correlation_dict = compute_sorted_correlations(df_encoded)

from pprint import pprint
display(correlation_dict[:2])  # preview first 2 entries
```

Show hidden output

```
df_encoded.columns
```

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
       'workplace', 'religion', 'financial_condition', 'hometown', 'latitude',
```

```
       'longitude', 'reason', 'time', 'method', 'feels_like', 'temp_min',
       'temp_max', 'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg',
       'clouds_sky', 'weather_main', 'weather_description', 'suicide_day',
       'suicide_month', 'suicide_year', 'suicide_week', 'suicide_weekday'],
      dtype='object')
```

```
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from tqdm import tqdm
import pandas as pd
import numpy as np
from typing import Optional, Dict, Tuple, List, Union
from copy import deepcopy

# Start with a deep copy of the dataset so we don't affect the original
df_imputed_forest_context = deepcopy(df_encoded)
df_imputed_forest_no_context = deepcopy(df_encoded)

# === Define constraints ===
constraints = {
    'age': (0, 100),
    'latitude': (20.57, 26.63),           # Bangladesh latitude range
    'longitude': (88.0167, 92.6833),      # Bangladesh longitude range
    'feels_like': (2.6, 300.65),          # Temperature range in Kelvin (lowest and highest)
    'temp_min': (2.6, 300.65),             # Same temp range
    'temp_max': (2.6, 300.65),
    'air_pressure': (1000, 1050),          # ~1000 hPa to a bit above 1020 hPa
    'air_humidity': (70, 100),              # Percent
    'wind_speed': (3, 20),                 # ~3 to 16+ km/h, added small buffer
    'wind_deg': (0, 360),                  # Wind direction in degrees, full circle
    'clouds_sky': (0, 100),                # Cloudiness in percent
    'time_encoded': (0, 23),               # Assuming time encoded as hour in 24h format
    'suicide_day': (1, 31),                # Day of month
    'suicide_month': (1, 12),              # Month number
    'suicide_year': (2017, 2025),          # Year range you gave
    'suicide_week': (1, 52),               # Week number (approximate)
    'suicide_weekday_encoded': (0, 6),
    'age_group_encoded': (0, 6),
    'financial_condition_encoded': (0, 4),
    'weather_main_encoded': (0, 7),
}
```

```
def impute_with_rf_context(
    df: pd.DataFrame,
    correlation_info,  # can be dict or list of dicts with 'column_name' and 'related_columns'
    constraints: Optional[Dict[str, Tuple[Union[int, float], Union[int, float]]]] = None,
    n_estimators: int = 100,
    random_state: int = 42,
    verbose: bool = True
) -> pd.DataFrame:

    def fill_missing(series: pd.Series) -> pd.Series:
        if series.isnull().any():
            if np.issubdtype(series.dtype, np.number):
                return series.fillna(series.mean())
            else:
```

```
                mode_val = series.mode()
                return series.fillna(mode_val.iloc[0] if not mode_val.empty else "missing")
        return series

    # Normalize correlation_info into list of entries
    if isinstance(correlation_info, dict):
        context_list = [
            {'column_name': col, 'related_columns': [{'related_column': rc} for rc in cols]}
            for col, cols in correlation_info.items()
        ]
    elif isinstance(correlation_info, list):
        context_list = correlation_info
    else:
        raise ValueError("correlation_info must be a dict or a list of dicts")

    for entry in tqdm(context_list, desc="RF Context Imputation"):
        col = entry['column_name']
        missing_mask = df[col].isna()
        if missing_mask.sum() == 0:
            continue

        is_categorical = df[col].dtype == 'object' or df[col].dtype.name == 'category'
        context_cols = [r['related_column'] for r in entry['related_columns'] if r['related_column'] in df.columns and r['related_column'] != col]
        context_cols = [c for c in context_cols if df[c].notnull().any()]

        if not context_cols:
            if verbose:
                print(f"⚠️ Skipping '{col}': no usable context columns.")
            continue

        df_known = df.loc[df[col].notnull()]
        df_missing = df.loc[df[col].isnull()]
        if df_known.empty or df_missing.empty:
            if verbose:
                print(f"⚠️ Skipping '{col}': no known or missing data.")
            continue

        X_train = df_known[context_cols].apply(fill_missing)
        y_train = df_known[col]
        X_pred = df_missing[context_cols].apply(fill_missing)

        model = RandomForestClassifier(n_estimators=n_estimators, random_state=random_state) if is_categorical else \
                RandomForestRegressor(n_estimators=n_estimators, random_state=random_state)

        try:
            model.fit(X_train, y_train)
            y_pred = model.predict(X_pred)
            df.loc[df_missing.index, col] = y_pred

            if constraints and col in constraints and not is_categorical:
                min_val, max_val = constraints[col]
                df[col] = df[col].clip(lower=min_val, upper=max_val)

            if verbose:
                print(f"✅ Imputed '{col}' using context columns: {context_cols}")
```

```
            except Exception as e:
                if verbose:
                    print(f"❌ Error imputing '{col}': {e}")

    return df
```

```
# from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
# from tqdm import tqdm
# import numpy as np
# import pandas as pd
# from typing import Optional, Dict, Tuple, List, Union

# def custom_missforest_imputer(
#     df: pd.DataFrame,
#     correlation_dict: Optional[Dict[str, List[str]]] = None,
#     constraints: Optional[Dict[str, Tuple[Union[int, float], Union[int, float]]]] = None,
#     n_estimators: int = 100,
#     random_state: int = 42,
#     verbose: bool = True
# ) -> pd.DataFrame:


#     def fill_missing_context(series: pd.Series) -> pd.Series:
#         if series.isnull().any():
#             if np.issubdtype(series.dtype, np.number):
#                 return series.fillna(series.mean())
#             else:
#                 # mode might be empty, handle that:
#                 mode_val = series.mode()
#                 if not mode_val.empty:
#                     return series.fillna(mode_val.iloc[0])
#                 else:
#                     # fallback: fill with a placeholder (e.g. string "missing" for categorical)
#                     return series.fillna("missing")
#         else:
#             return series

#     columns_to_impute = [col for col in df.columns if df[col].isnull().any()]

#     for col in tqdm(columns_to_impute, desc="Imputing columns"):
#         is_categorical = df[col].dtype == 'object' or df[col].dtype.name == 'category'

#         # Choose context columns
#         if correlation_dict and col in correlation_dict:
#             context_cols = [c for c in correlation_dict[col] if c in df.columns and c != col]
#         else:
#             context_cols = [c for c in df.columns if c != col]

#         # Remove context columns that are completely NaN
#         context_cols = [c for c in context_cols if df[c].notnull().any()]
#         if not context_cols:
#             if verbose:
#                 print(f"⚠️ Skipping {col}: No usable context columns.")
#             continue
```

```
#         df_known = df.loc[df[col].notnull()]
#         df_missing = df.loc[df[col].isnull()]

#         if df_known.empty or df_missing.empty:
#             if verbose:
#                 print(f"⚠️ Skipping {col}: No known or missing data found.")
#             continue

#         X_train = df_known[context_cols].copy()
#         y_train = df_known[col]
#         X_pred = df_missing[context_cols].copy()

#         # Fill missing context in both train and predict sets
#         for context_col in context_cols:
#             X_train[context_col] = fill_missing_context(X_train[context_col])
#             X_pred[context_col] = fill_missing_context(X_pred[context_col])

#         if X_train.empty or X_pred.empty:
#             if verbose:
#                 print(f"⚠️ Skipping {col}: Insufficient data after filling context.")
#             continue

#         model = RandomForestClassifier(n_estimators=n_estimators, random_state=random_state) if is_categorical else \
#                 RandomForestRegressor(n_estimators=n_estimators, random_state=random_state)

#         try:
#             model.fit(X_train, y_train)
#             y_pred = model.predict(X_pred)

#             df.loc[X_pred.index, col] = y_pred

#             if constraints and col in constraints and not is_categorical:
#                 min_val, max_val = constraints[col]
#                 df[col] = df[col].clip(lower=min_val, upper=max_val)

#         except Exception as e:
#             if verbose:
#                 print(f"❌ Error imputing '{col}': {e}")

#     return df
```

```
# df_imputed_forest_context = impute_with_rf_context(df_imputed_forest_context, correlation_dict=correlation_dict, constraints=constraints)

df_imputed_forest_context = impute_with_rf_context(df_imputed_forest_context, correlation_info=correlation_dict, constraints=constraints)
```

Show hidden output

```
missing_data_report(df_imputed_forest_context)
```

Show hidden output

```
def impute_with_rf_no_context(
    df: pd.DataFrame,
    constraints: Optional[Dict[str, Tuple[Union[int, float], Union[int, float]]]] = None,
```

```
        n_estimators: int = 100,
        random_state: int = 42,
        verbose: bool = True
    ) -> pd.DataFrame:
        """
        Random Forest imputation using all other columns as predictors.
        Fully aligned with standard MICE best combo structure.
        """

        def fill_missing(series: pd.Series) -> pd.Series:
            if series.isnull().any():
                if np.issubdtype(series.dtype, np.number):
                    return series.fillna(series.mean())
                else:
                    mode_val = series.mode()
                    return series.fillna(mode_val.iloc[0] if not mode_val.empty else "missing")
            return series

        # Iterate over columns with missing values
        for col in tqdm([c for c in df.columns if df[c].isnull().any()], desc="RF No-Context Imputation"):
            missing_mask = df[col].isna()
            if missing_mask.sum() == 0:
                continue

            is_categorical = df[col].dtype == 'object' or df[col].dtype.name == 'category'

            # All other columns as predictors, excluding fully-NaN
            predictors = [c for c in df.columns if c != col and df[c].notnull().any()]
            if not predictors:
                if verbose:
                    print(f"⚠️ Skipping '{col}': no usable predictors.")
                continue

            df_known = df.loc[df[col].notnull()]
            df_missing = df.loc[df[col].isnull()]
            if df_known.empty or df_missing.empty:
                if verbose:
                    print(f"⚠️ Skipping '{col}': no known or missing data.")
                continue

            X_train = df_known[predictors].apply(fill_missing)
            y_train = df_known[col]
            X_pred = df_missing[predictors].apply(fill_missing)

            model = RandomForestClassifier(n_estimators=n_estimators, random_state=random_state) if is_categorical else \
                    RandomForestRegressor(n_estimators=n_estimators, random_state=random_state)

            try:
                model.fit(X_train, y_train)
                y_pred = model.predict(X_pred)
                df.loc[df_missing.index, col] = y_pred

                # Apply constraints to numeric columns
                if constraints and col in constraints and not is_categorical:
                    min_val, max_val = constraints[col]
                    df[col] = df[col].clip(lower=min_val, upper=max_val)
```

```
            if verbose:
                print(f"✅ Imputed '{col}' using all columns as predictors")

        except Exception as e:
            if verbose:
                print(f"❌ Error imputing '{col}': {e}")

    return df
```

```
# def impute_with_knn_no_context(df, constraints=None, n_neighbors=5):
#     # Copy to avoid modifying original
#     df_result = df.copy()

#     try:
#         imputer = KNNImputer(n_neighbors=n_neighbors)
#         imputed_array = imputer.fit_transform(df_result)

#         # Put back imputed values
#         df_imputed = pd.DataFrame(imputed_array, columns=df_result.columns, index=df_result.index)

#         # Apply constraints if provided
#         if constraints:
#             for col, (min_val, max_val) in constraints.items():
#                 if col in df_imputed.columns:
#                     df_imputed[col] = df_imputed[col].clip(lower=min_val, upper=max_val)

#         print("✅ Imputation without context done.")
#         return df_imputed

#     except Exception as e:
#         print(f"❌ Error during KNN imputation without context: {e}")
#         return df_result  # return original in case of failure
```

```
df_imputed_rf_no_context = impute_with_rf_no_context(df_imputed_forest_no_context, constraints=constraints)
```

```
RF No-Context Imputation:   3%|▎         | 1/29 [00:02<01:02,  2.24s/it]✅ Imputed 'age' using all columns as predictors
RF No-Context Imputation:   7%|▋         | 2/29 [00:03<00:47,  1.75s/it]✅ Imputed 'age_group' using all columns as predictors
RF No-Context Imputation:  10%|█         | 3/29 [00:04<00:39,  1.54s/it]✅ Imputed 'gender' using all columns as predictors
RF No-Context Imputation:  14%|█▍        | 4/29 [00:05<00:33,  1.35s/it]✅ Imputed 'marital_status' using all columns as predictors
RF No-Context Imputation:  17%|█▋        | 5/29 [00:07<00:33,  1.38s/it]✅ Imputed 'profession_group' using all columns as predictors
RF No-Context Imputation:  21%|██        | 6/29 [00:08<00:27,  1.19s/it]✅ Imputed 'workplace' using all columns as predictors
RF No-Context Imputation:  24%|██▍       | 7/29 [00:09<00:29,  1.34s/it]✅ Imputed 'religion' using all columns as predictors
RF No-Context Imputation:  28%|██▊       | 8/29 [00:10<00:24,  1.18s/it]✅ Imputed 'financial_condition' using all columns as predictors
RF No-Context Imputation:  31%|███       | 9/29 [00:12<00:24,  1.22s/it]✅ Imputed 'hometown' using all columns as predictors
RF No-Context Imputation:  34%|███▍      | 10/29 [00:13<00:23,  1.25s/it]✅ Imputed 'latitude' using all columns as predictors
RF No-Context Imputation:  38%|███▊      | 11/29 [00:15<00:25,  1.39s/it]✅ Imputed 'longitude' using all columns as predictors
RF No-Context Imputation:  41%|████      | 12/29 [00:17<00:26,  1.55s/it]✅ Imputed 'reason' using all columns as predictors
RF No-Context Imputation:  45%|████▍     | 13/29 [00:18<00:25,  1.60s/it]✅ Imputed 'time' using all columns as predictors
RF No-Context Imputation:  48%|████▊     | 14/29 [00:20<00:25,  1.72s/it]✅ Imputed 'method' using all columns as predictors
RF No-Context Imputation:  52%|█████▏    | 15/29 [00:21<00:21,  1.51s/it]✅ Imputed 'feels_like' using all columns as predictors
RF No-Context Imputation:  55%|█████▌    | 16/29 [00:22<00:18,  1.43s/it]✅ Imputed 'temp_min' using all columns as predictors
RF No-Context Imputation:  59%|█████▊    | 17/29 [00:24<00:16,  1.38s/it]✅ Imputed 'temp_max' using all columns as predictors
RF No-Context Imputation:  62%|██████▏   | 18/29 [00:25<00:15,  1.39s/it]✅ Imputed 'air_pressure' using all columns as predictors
RF No-Context Imputation:  66%|██████▌   | 19/29 [00:26<00:13,  1.33s/it]✅ Imputed 'air_humidity' using all columns as predictors
RF No-Context Imputation:  69%|██████▉   | 20/29 [00:28<00:13,  1.55s/it]✅ Imputed 'wind_speed' using all columns as predictors
RF No-Context Imputation:  72%|███████▏  | 21/29 [00:30<00:11,  1.46s/it]✅ Imputed 'wind_deg' using all columns as predictors
```

```
RF No-Context Imputation:  76%|███████   | 22/29 [00:31<00:09,  1.32s/it]✅ Imputed 'clouds_sky' using all columns as predictors
RF No-Context Imputation:  79%|███████   | 23/29 [00:31<00:06,  1.04s/it]✅ Imputed 'weather_main' using all columns as predictors
RF No-Context Imputation:  83%|███████▍  | 24/29 [00:32<00:04,  1.14it/s]✅ Imputed 'weather_description' using all columns as predictors
RF No-Context Imputation:  86%|███████▊  | 25/29 [00:34<00:05,  1.29s/it]✅ Imputed 'suicide_day' using all columns as predictors
RF No-Context Imputation:  90%|████████  | 26/29 [00:35<00:03,  1.21s/it]✅ Imputed 'suicide_month' using all columns as predictors
RF No-Context Imputation:  93%|████████▍ | 27/29 [00:36<00:02,  1.22s/it]✅ Imputed 'suicide_year' using all columns as predictors
RF No-Context Imputation:  97%|████████▋ | 28/29 [00:37<00:01,  1.28s/it]✅ Imputed 'suicide_week' using all columns as predictors
RF No-Context Imputation: 100%|█████████| 29/29 [00:40<00:00,  1.39s/it]✅ Imputed 'suicide_weekday' using all columns as predictors
```

```
missing_data_report(df_imputed_forest_no_context)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

Group 2: Columns with >40% to ≤70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

Group 3: Columns with >20% to ≤40% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

Group 4: Columns with ≤20% missing data
              Column  Missing % Data Type
                 age        0.0   float64
           age_group        0.0   float64
              gender        0.0   float64
      marital_status        0.0   float64
    profession_group        0.0   float64
           workplace        0.0   float64
            religion        0.0   float64
financial_condition        0.0   float64
            hometown        0.0   float64
            latitude        0.0   float64
           longitude        0.0   float64
              reason        0.0   float64
                time        0.0   float64
              method        0.0   float64
          feels_like        0.0   float64
            temp_min        0.0   float64
            temp_max        0.0   float64
        air_pressure        0.0   float64
        air_humidity        0.0   float64
          wind_speed        0.0   float64
            wind_deg        0.0   float64
          clouds_sky        0.0   float64
        weather_main        0.0   float64
weather_description        0.0   float64
         suicide_day        0.0   float64
       suicide_month        0.0   float64
        suicide_year        0.0   float64
        suicide_week        0.0   float64
     suicide_weekday        0.0   float64
```

```
import json
import numpy as np
import pandas as pd
import os

def round_imputed_categoricals(df_non_encoded, df_imputed, label_json_path):
    """
    Rounds imputed values of label-encoded categorical columns to nearest integer
    and clips to valid label range based on JSON label info.

    Parameters:
    - df_non_encoded: Original non-encoded DataFrame
    - df_imputed: Imputed DataFrame (floats from MissForest)
    - label_json_path: Path to label_encoders.json

    Returns:
    - df_rounded: DataFrame with categorical columns safely rounded
    """

    # Load label JSON
    with open(label_json_path, 'r') as f:
        label_encoders_serializable = json.load(f)

    df_rounded = df_imputed.copy()

    # Get categorical columns from original non-encoded DataFrame
    cat_cols = df_non_encoded.select_dtypes(include='object').columns

    for col in cat_cols:
        if col in df_rounded.columns and col in label_encoders_serializable:
            # Get valid range for this label-encoded column
            n_labels = len(label_encoders_serializable[col])
            min_val, max_val = 0, n_labels - 1

            # Round to nearest integer and clip to valid range
            df_rounded[col] = np.round(df_rounded[col]).astype(int)
            df_rounded[col] = df_rounded[col].clip(min_val, max_val)

    return df_rounded


drive_path = '/content/drive/MyDrive/capstone/'
label_json_path = drive_path + 'label_encoders.json'
```

```
df_imputed_forest_context = round_imputed_categoricals(df_non_encoded=df_non_encoded, df_imputed=df_imputed_forest_context, label_json_path=label_json_path)
df_imputed_forest_context.tail(5)
```

| | age | age_group | gender | marital_status | profession_group | workplace | religion | financial_condition | hometown | latitude | ... | wind_speed | wind_deg | clouds_sky | weather_ma |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **1612** | 15.00 | 5 | 0 | 5 | 26 | 0 | 5 | 1 | 77 | 23.781767 | ... | 4.8497 | 221.95 | 35.02 | |
| **1613** | 27.00 | 0 | 0 | 6 | 26 | 2 | 5 | 3 | 71 | 23.706342 | ... | 4.9188 | 177.74 | 65.65 | |
| **1614** | 9.00 | 2 | 1 | 5 | 25 | 1 | 5 | 2 | 71 | 23.708136 | ... | 8.4030 | 240.55 | 70.68 | |
| **1615** | 21.19 | 6 | 0 | 5 | 26 | 0 | 5 | 1 | 76 | 23.898795 | ... | 8.0000 | 246.15 | 27.32 | |
| **1616** | 26.00 | 0 | 0 | 3 | 14 | 0 | 4 | 2 | 3 | 22.655478 | ... | 3.8000 | 197.19 | 68.64 | |

5 rows × 29 columns

```
df_imputed_forest_no_context = round_imputed_categoricals(df_non_encoded=df_non_encoded, df_imputed=df_imputed_forest_no_context, label_json_path=label_json_path)
df_imputed_forest_no_context.tail(5)
```

| | age | age_group | gender | marital_status | profession_group | workplace | religion | financial_condition | hometown | latitude | ... | wind_speed | wind_deg | clouds_sky | weather_ma |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **1612** | 15.00 | 5 | 0 | 5 | 26 | 0 | 5 | 1 | 76 | 23.759420 | ... | 5.461 | 231.15 | 32.75 | |
| **1613** | 27.00 | 0 | 0 | 6 | 26 | 2 | 5 | 3 | 69 | 23.711043 | ... | 3.000 | 187.87 | 61.52 | |
| **1614** | 9.00 | 2 | 1 | 5 | 25 | 1 | 5 | 2 | 70 | 23.726419 | ... | 7.195 | 234.23 | 71.70 | |
| **1615** | 21.11 | 6 | 0 | 5 | 26 | 0 | 5 | 1 | 76 | 23.898795 | ... | 8.000 | 244.03 | 28.37 | |
| **1616** | 26.00 | 0 | 0 | 3 | 14 | 0 | 5 | 2 | 3 | 22.655478 | ... | 3.800 | 195.59 | 68.95 | |

5 rows × 29 columns

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format

# # Make the latest DataFrame downloadable
# from google.colab import files

# # Specify the filename for the Excel file
# filename1 = '10_forest_context.xlsx'

# # Save the DataFrame to an Excel file
# df_imputed_forest_context.to_excel(filename1, index=False)

# # Download the file
# files.download(filename1)
```

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format

# # Make the latest DataFrame downloadable
# from google.colab import files

# # Specify the filename for the Excel file
# filename2 = '11_forest_no_context.xlsx'

# # Save the DataFrame to an Excel file
# df_imputed_forest_no_context.to_excel(filename2, index=False)
```

```
# # Download the file
# files.download(filename2)
```